Name: _____

5343 Advanced Algorithms

Exam 02 – Spring 2026

Instructions.

- Pencil only.
- Put effort into your writing; use space wisely and align text for maximum readability.
- If you have a question, walk down and ask.
- Any visible phones or other devices are grounds for failure.

Section	**Points Possible**	**Points Earned**
I. Multiple Choice (12×2)	24	
II. Fill in the Blank (10×2)	20	
III. Short Answer	86	
IV. Challenge (instructor discretion)	—	
Total	**130**	

Section I Multiple Choice (2 points each)

Circle the single best answer for each question.

[2] 1. Which container guarantees $O(1)$ access to the minimum (or maximum) element?

A. Binary Search Tree

B. Sorted Array

C. Min-Heap (or Max-Heap)

D. Hash Table

A heap maintains the heap property so the root is always the extreme element. Access is $O(1)$; removal is $O(\log n)$.

[2] 2. Binary search requires which *combination* of preconditions?

A. Sorted data only

B. $O(1)$ random access by index only

C. Sorted data and $O(1)$ random access by index

D. A hash function and sorted data

Binary search computes `mid` and jumps directly to it each step. Without $O(1)$ index access (e.g., on a linked list), reaching `mid` costs $O(n)$, making the overall algorithm no faster than linear search.

[2] 3. A hash table using linear probing has load factor $\alpha = 0.90$. Which statement best describes expected performance?

A. $O(1)$ — the load factor has no effect on expected lookup time

B. Severely degraded — expected unsuccessful probes ≈ 50

C. $O(\log n)$ — similar to a balanced BST

D. Constant 2.5 probes regardless of load

Expected unsuccessful probes under linear probing: $\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$. At $\alpha = 0.90$: $\frac{1}{2}(1 + 100) = 50.5$. The table is nearly full; clustering dominates.

[2] 4. Which sorting algorithm is **not** stable?

A. Merge sort

B. Heapsort

C. Timsort

D. Insertion sort

Heapsort's heap-restructuring steps may reorder equal elements. Merge sort, Timsort, and insertion sort all preserve relative order of equal keys.

[2] 5. You examine a trace of an MST algorithm. The *first* edge added is **not** the globally minimum-weight edge in the graph. This trace must belong to:

A. Kruskal's algorithm

B. Prim's algorithm

C. Either algorithm — both can select a non-minimum first edge

D. Neither — both always select the global minimum first

Kruskal's processes edges in globally sorted order, so its first selection is always the global minimum. If the first edge is not the global minimum, the algorithm must be Prim's, which picks the cheapest edge incident to its growing tree — not necessarily the cheapest in the entire graph.

[2] 6. When inserting into a Red-Black tree, the parent is red and the uncle is **also red**. The correct fix is:

A. Single rotation at the grandparent

B. Double rotation at the parent

C. Recolor parent and uncle to black; recolor grandparent to red; check upward

D. Recolor only the new node to black

When the uncle is red, no rotation is needed. Flipping parent and uncle to black preserves the black-height on both sides. Flipping the grandparent to red passes the potential violation upward for recursive resolution.

[2] 7. A Ternary Search Tree (TST) is preferred over a standard Trie primarily when:

A. The alphabet is large and the key set is sparse

B. Only exact-match lookups are needed

C. Memory is unlimited

D. The key set is static and fully known in advance

A standard trie allocates $|\Sigma|$ child pointers at every node, most null for sparse alphabets. A TST uses exactly 3 pointers per node regardless of $|\Sigma|$, making it far more memory-efficient when the alphabet is large or the key distribution is uneven.

[2] 8. DFS on a **graph** requires one additional bookkeeping structure that tree DFS does not. What is it?

A. A priority queue

B. A visited set

C. A sorted edge list

D. A parent-pointer array

Trees cannot have cycles, so recursive tree DFS terminates naturally. Graphs can have cycles; without tracking visited nodes, DFS would loop forever.

[2] 9. Heapify — building a min-heap from an arbitrary unsorted array of n elements — runs in:

A. $O(n \log n)$

B. $O(n)$

C. $O(n^2)$

D. $O(\log n)$

The naive bound of $O(n \log n)$ overcounts. Most nodes are near the bottom and sift down only a few levels. The exact sum across all nodes forms a convergent series that resolves to $O(n)$.

[2] 10. A hash table uses separate chaining. Which statement is true once the load factor exceeds 1?

A. The table continues to function; chains grow longer

B. Every insertion triggers an automatic resize

C. The table enters an error state; load factor cannot exceed 1

D. All lookups degrade to $O(n)$ regardless of the hash function

Chaining stores elements in linked lists attached to each slot, so there is no "full table" condition. $\alpha > 1$ means chains average more than one element, degrading performance linearly — but the structure remains correct. Open addressing (not chaining) breaks when $\alpha \geq 1$.

[2] 11. A perfect hash function guarantees:

A. $O(1)$ expected lookup for any dynamic key set

B. $O(1)$ **worst-case lookup for a static, fully known key set with zero collisions**

C. Better cache performance than open addressing in all cases

D. No need for a secondary hash function

A perfect hash maps every key in a fixed, known set to a distinct slot with no collisions — $O(1)$ worst-case, not just expected. It requires the key set to be static; it cannot adapt to keys inserted at runtime.

[2] 12. Which statement about inorder traversal is correct?

A. Inorder traversal on any binary tree produces sorted output

B. Inorder traversal produces sorted output only when applied to a BST

C. Inorder traversal is equivalent to BFS on a tree

D. Inorder traversal requires a visited set to prevent revisiting nodes

Inorder (left–root–right) exploits the BST ordering invariant. On a general binary tree with no ordering invariant, the output order is arbitrary.

Section II Fill in the Blank (2 points each)

Write the word, phrase, or expression that correctly completes each statement. Be precise.

[2] 13. The load factor of a hash table is $\alpha =$ _____, where n is the number of stored elements and m is the table capacity.

$\alpha = n/m$

[2] 14. Building a heap from an arbitrary unsorted array in-place (heapify) runs in $O($_____$)$ time — *not* $O(n \log n)$.

$O(n)$

[2] 15. Kruskal's algorithm uses _____ to detect whether adding an edge would form a cycle.

Union-Find (also accepted: Disjoint Set Union / DSU)

[2] 16. When inserting a new node into a Red-Black tree, the node is initially colored _____.

Red

[2] 17. The four edit operations used to generate candidate corrections in a classic spell checker are: _____, _____, _____, and _____.

Substitution, Insertion, Deletion, Transposition (swap of adjacent characters)

[2] 18. A _____ tree compresses chains of single-child trie nodes into edges labeled with substrings, reducing node count from $O(\text{total characters})$ to $O(n)$ while preserving all prefix relationships.

Radix (tree)

[2] 19. In double hashing, the second hash function $h_2(k)$ must never return _____, because that stride would cause the probe sequence to revisit the same slot indefinitely.

0 (zero)

[2] 20. A Red-Black tree guarantees a maximum height of _____$\log n$.

2

[2] 21. Huffman coding always produces a _____ binary tree, meaning every internal node has exactly two children.

Full binary tree. ("Complete" is incorrect — the tree is full but not necessarily complete.)

[2] 22. T9 predictive text stores its dictionary in a _____ so that, for each key press, entire subtrees whose root character falls outside the pressed key's character set can be pruned immediately.

Trie

Short Answer Format Guide

The questions in Section III expect a specific level of detail. The worked examples below show the **exact grammar and notation** you should use. Each example uses a small throwaway graph — not the graphs on your exam.

Kruskal's Algorithm — Example Format

Graph: vertices A, B, C; edges A–B (4), A–C (1), B–C (2).

Step 1 — Sort all edges by weight:

Edge	Weight
A–C	1
B–C	2
A–B	4

Step 2 — Process edges in order (Union-Find decides Add or Skip):

Step	Edge	Wt.	Action	Components after
1	A–C	1	Add	{A,C} {B}
2	B–C	2	Add	{A,B,C}
3	A–B	4	Skip	{A,B,C} *(A and B share a component — cycle)*

MST edges: A–C (1), B–C (2). **Total weight:** 3.

Prim's Algorithm — Example Format

*Same graph; start at vertex **A**. key[v] = cheapest known edge connecting v to the current tree.*

Step 0 — Initialize: Set key[A] = 0, all others = ∞. Extract A (no edge added; A is the seed). Examine A's neighbors and update their keys:

B: $\infty \to 4$ (edge A–B, weight 4)
C: $\infty \to 1$ (edge A–C, weight 1)

Steps 1–2 — Extract the minimum-key vertex, add its edge, update neighbors:

Step	Extract	Edge Added	Wt.	Key updates
1	C	A–C	1	B: $4 \to 2$ (C–B = 2 is cheaper than A–B = 4)
2	B	C–B	2	— (no unvisited neighbors remain)

MST edges: A–C (1), C–B (2). **Total weight:** 3.

Key update grammar: write `vertex: old → new (via edge X--Y = weight)`. If no update occurs, write `---`. If a neighbor is already in the MST, skip it silently.

Huffman Tree — Example Format

Alphabet: X (freq 3), Y (freq 2), Z (freq 1).

Step 1 — Build the min-heap merge trace:

Step	Left (weight)	Right (weight)	New node weight
1	Z (1)	Y (2)	ZY (3)
2	X (3)	ZY (3)	root (6)

Always extract the two lowest-weight nodes from the heap at each step. After merging, reinsert the new node and repeat.

Step 2 — Draw the tree (0 = left edge, 1 = right edge):

```
root (6)
  0 → X (3)
  1 → ZY (3)
      0 → Z (1)
      1 → Y (2)
```

Step 3 — Read codes top-down (concatenate edge labels on path from root to leaf):

Char	Freq	Code	Length	Bits
X	3	0	1	3
Z	1	10	2	2
Y	2	11	2	4
			Total (Huffman):	9
			Total (fixed 2-bit):	12

Verify prefix-free: no code is a prefix of another. Verify optimality: higher-frequency symbols have shorter codes.

Section III Short Answer

[10] 23. **Container / Algorithm Mismatch Diagnosis.**

A student writes the following code to process a stream of one million sensor readings, flagging each reading the first time it appears:

```
seen = []
for reading in sensor_stream:        # 1,000,000 readings
    if reading not in seen:          # membership check
        seen.append(reading)
        flag(reading)
```

[4] (a) Identify the container, the operation being performed, and the actual time cost of that operation on this container.

Container: Python list (dynamic array). **Operation:** Membership check (`in`). **Cost:** $O(n)$ linear scan — Python compares each element from the front until it finds a match or exhausts the list.

[3] (b) What is the total worst-case cost of the loop over all 10^6 readings? Explain why.

Each of the $n = 10^6$ iterations performs an $O(n)$ membership check. Total: $O(n^2)$ — the same asymptotic cost as a nested loop over the entire stream.

[3] (c) Name the container that fixes this mismatch and state the corrected total cost.

Replace `seen = []` with `seen = set()`. A Python set is backed by a hash table; `in` is $O(1)$ average. Total loop cost drops to $O(n)$.

[10] 24. **Hash Table — Linear Probing Trace.**

A hash table has $m = 11$ slots (indices 0–10). The hash function is $h(k) = k \bmod 11$. Insert the keys **22, 35, 15, 44, 55, 26** in that order.

[6] (a) Complete the table. For each key, show its initial hash slot and the final slot after collision resolution under linear probing.

Key	$h(k)$	**Probe sequence (slots tried)**	**Final Slot**
22			
35			
15			
44			
55			
26			

Key	$h(k)$	Probe sequence	Final Slot
22	0	0 (empty)	0
35	2	2 (empty)	2
15	4	4 (empty)	4
44	0	0 (22) $\rightarrow$ 1 (empty)	1
55	0	0 (22) $\rightarrow$ 1 (44) $\rightarrow$ 2 (35) $\rightarrow$ 3 (empty)	3
26	4	4 (15) $\rightarrow$ 5 (empty)	5

[2] (b) Draw the final table state. Mark empty slots with `---`.

Slot	Contents
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Slot 0: 22 Slot 1: 44 Slot 2: 35 Slot 3: 55 Slot 4: 15 Slot 5: 26 Slots 6–10: —

[2] (c) Name the clustering phenomenon visible in this trace and explain it in one sentence.

Primary clustering: keys hashing to different initial slots (0, 2, 4) pile into the same contiguous run (slots 0–5) because each collision extends the occupied block, forcing subsequent insertions to probe further.

[12] 25. **AVL Tree — Insertions and Rotations.**

Insert the following keys into an initially empty AVL tree **in the order given**:

$$10, \quad 20, \quad 30, \quad 15, \quad 5, \quad 25, \quad 28$$

[4] (a) After inserting 10, 20, and 30, the tree is unbalanced. Identify the imbalanced node, state the rotation case (e.g., Right-Right), and draw the tree after the fix. Label each node's balance factor.

After inserting 30, node **10** has $bf = -2$ (right-heavy); its right child 20 has $bf = -1$ (also right-heavy) — this is the **Right-Right** case.

Fix: single **left rotation** at node 10.

Result: 20 is the new root ($bf = 0$), with 10 as left child ($bf = 0$) and 30 as right child ($bf = 0$).

[4] (b) Insert 15, 5, then 25. Draw the tree after all three insertions and label each node's balance factor. (No rotations occur during these three insertions.)

After inserting 15: node 10 gains right child 15 ($bf = -1$); node 20 becomes $bf = +1$.
After inserting 5: node 10 gains left child 5 ($bf = 0$); node 20 becomes $bf = 0$.
After inserting 25: node 30 gains left child 25 ($bf = +1$); node 20 becomes $bf = -1$.

Final tree ($|bf| \leq 1$ everywhere — no violation):

`20(bf=-1) / left:  10(bf=0, left:5, right:15) / right:  30(bf=+1, left:25)`

[4] (c) Insert 28. Identify the imbalanced node and its balance factor, name the rotation case, and show each rotation explicitly. Draw the final balanced tree.

After inserting 28 as right child of 25:
Node **30** has $bf = +2$ (left-heavy); its left child 25 has $bf = -1$ (right-heavy). This is the **Right-Left** case (zig-zag).

Step 1 — Left rotate at 25: 28 rises; 25 drops to become 28's left child. Now 30's left child is 28 (with 25 as 28's left child): $bf(28) = +1$, $bf(30) = +2$. Shape is now Left-Left relative to 30.

Step 2 — Right rotate at 30: 28 rises to replace 30; 30 drops to become 28's right child.

Final subtree: root **28** ($bf = 0$), left child 25 ($bf = 0$), right child 30 ($bf = 0$).
Full tree: `20(bf=-1) / left:10(bf=0) / right:28(bf=0, left:25, right:30)`
(Node 10 still has children 5 and 15.)

[12] 26. **Red-Black Tree — Insertion and Fix.**

Insert the keys **10, 20, 30, 15** into an initially empty Red-Black tree, in that order.

For each insertion, state (a) whether a fix is required, and if so, (b) the fix type (recolor / single rotation + recolor / double rotation + recolor), and (c) the resulting tree with node colors labeled **(B)** or **(R)**.

Insert 10: Empty tree; 10 becomes root. By Property 2, color **black**.
Tree: `10(B)`

Insert 20: New node 20 is red. Parent 10 is **black** — no violation.
Tree: `10(B), right child:  20(R)`

Insert 30: New node 30 is red. Parent 20 is red — **violation**. Uncle = left child of 10 = null (black). Uncle is black, parent is a right child, new node is a right child: **Right-Right** case.
Fix: single **left rotation** at 10, then swap colors of 10 and 20.
Result: 20(**B**) is new root; 10(**R**) is left child; 30(**R**) is right child.

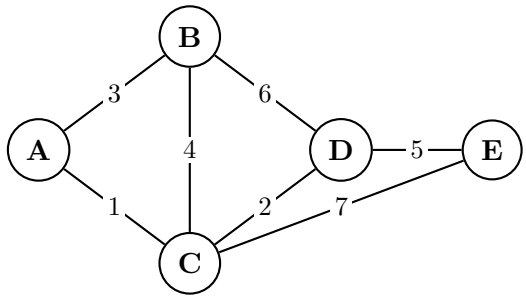
Insert 15: New node 15 is red. Parent 10 is red — **violation**. Uncle = right child of 20 = 30(**R**) — uncle is **red**.
Fix: **recolor** — parent 10 → black; uncle 30 → black; grandparent 20 → red. But 20 is the root — recolor root back to **black**.
Result: 20(**B**) / left: 10(**B**, right child: 15(**R**)) / right: 30(**B**)

Using your final tree, verify all five Red-Black properties. Mark each as holds or violated.

#	**Property**	**Holds / Violated?**
1	Every node is red or black.	Holds
2	The root is black.	Holds — 20 is black
3	All null (leaf) nodes are black.	Holds (by convention)
4	A red node's children are both black (no two consecutive red nodes).	Holds — 15(R)'s parent is 10(B)
5	Every root-to-null path has the same number of black nodes (black-height).	Holds — all paths have black-height 2

[12] 27. **Minimum Spanning Tree — Kruskal's Algorithm.**

Consider the undirected, weighted graph below.



[6] (a) Trace Kruskal's algorithm. Process edges in globally sorted order. For each edge, state the action (Add / Skip) and the component state after. The initial component state is $ABCDE$ as you add components to the MST, you will show a new state like A B C D E− >A B C D E as you continue to add nodes to the MST.

Step	**Edge**	**Wt.**	**Action**	**Components after**
1				
2				
3				
4				
5				
6				
7				

Sorted edges: A–C(1), C–D(2), A–B(3), B–C(4), D–E(5), B–D(6), C–E(7)

Step	Edge	Wt	Action	Components
1	A–C	1	Add	{A,C} {B} {D} {E}
2	C–D	2	Add	{A,C,D} {B} {E}
3	A–B	3	Add	{A,B,C,D} {E}
4	B–C	4	Skip	{A,B,C,D} {E}
5	D–E	5	Add	{A,B,C,D,E}
6	B–D	6	Skip	{A,B,C,D,E}
7	C–E	7	Skip	{A,B,C,D,E}

MST edges: A–C, C–D, A–B, D–E. Total weight: $1 + 2 + 3 + 5 = \mathbf{11}$.

[3] (b) State the MST edges and total weight.

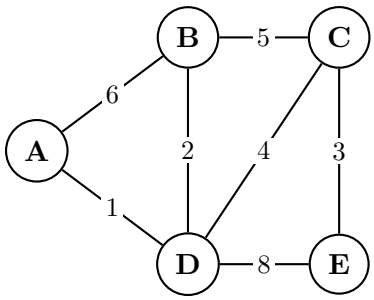
MST: {A–C (1), C–D (2), A–B (3), D–E (5)}. Total weight = **11**.

[3] (c) In one or two sentences, explain how Kruskal's avoids adding a cycle when it encounters the B–C edge at Step 4.

Before Step 4, B and C are already in the same component {A,B,C,D}. Kruskal's calls `find(B)` and `find(C)`; both return the same component root, so `find(B) == find(C)`. Adding B–C would connect two already-connected vertices, forming a cycle, so the edge is skipped.

[12] 28. **Minimum Spanning Tree — Prim's Algorithm.**

Consider the undirected, weighted graph below. (*Alternate version of SA 5.*)



[6] (a) Trace Prim's algorithm starting at vertex **A**. At each step, record the vertex extracted from the priority queue, the edge added to the MST, and any key (priority) updates made to remaining vertices.

Convention: key[v] = minimum known edge weight connecting v to the current tree. ∞ = not yet reached.

Step	**Extract**	**Edge Added**	**Wt.**	**Key updates (vertex: old → new)**
0 (init)	A	—	—	B: $\infty \to 6$, D: $\infty \to 1$
1				
2				
3				
4				

Step	Extract	Edge Added	Wt	Key updates
0	A	—	—	B: $\infty \to 6$ (via A), D: $\infty \to 1$ (via A)
1	D	A–D	1	B: $6 \to 2$ (D–B shorter), C: $\infty \to 4$ (via D), E: $\infty \to 8$ (via D)
2	B	D–B	2	C: 4 unchanged (B–C=5 > 4)
3	C	D–C	4	E: $8 \to 3$ (C–E shorter)
4	E	C–E	3	— (all vertices in MST)

[3] (b) State the MST edges and total weight.

MST: {A–D (1), D–B (2), C–E (3), D–C (4)}. Total weight = **10**.

[3] (c) Prim's avoids cycles without ever calling a `find()` operation. Explain in one to two sentences the mechanism that makes cycle detection unnecessary.

Prim's only considers edges that cross the frontier between the current tree and *unvisited* vertices. Once a vertex is extracted from the priority queue and added to the MST, it is marked visited and never reconsidered, so an edge back to an already-visited vertex can never be selected — cycles are structurally impossible in the candidate set.

[12] 29. **Trie Insertion and T9 Predictive Text.**

[5] (a) Insert the following words into a standard character trie:

`cat`, `car`, `card`, `care`, `can`, `bat`, `bad`, `band`

Draw the resulting trie in the space below. Mark each terminal node with a double circle or •. Shared prefixes must share edges and nodes.

[See instructor solution diagram.]

Root branches: `b`, `c`

`c`→a→t•, c→a→r•→d•, c→a→r→e•, c→a→n•

`b`→a→t•, b→a→d•, b→a→n→d•

[3] (b) List all words in the trie that share the prefix `ca`. State the retrieval cost in terms of P (prefix length) and K (number of matching words).

Words sharing prefix `ca`: **cat, car, card, care, can**.
Cost: $O(P+K)$ — $O(P)$ to walk down to the prefix node, then $O(K)$ to enumerate all terminal nodes in the subtree beneath it.

[4] (c) A user presses key sequence **2-2-6** on a T9 keypad (2 = ABC, 6 = MNO). Explain step-by-step how a trie-based T9 system narrows the candidate word set, and identify any matching words from the trie above.

Key sequence 2-2-6 maps to character sets: $\{A, B, C\} \to \{A, B, C\} \to \{M, N, O\}$.

Step 1: From root, follow only edges labelled with a character in $\{a, b, c\}$ — branches `b` and `c` survive; all others pruned.
Step 2: From `b` and `c`, follow only edges in $\{a, b, c\}$ — both lead to `a`, giving paths `ba` and `ca`.
Step 3: From `ba` and `ca`, follow only edges in $\{m, n, o\}$ — both lead to `n`, giving paths `ban` and `can`.

Terminal nodes at depth 3: `can`• exists in the trie. `ban` reaches `b`→`a`→`n` but that node is *not* terminal (only `band` terminates through it).

Match from the trie: `can`.

[12] 30. **Huffman Tree — Build and Encode.**

The frequency table below describes character counts in a compressed text segment.

Character	**Frequency**
A	12
B	7
C	5
D	4
E	3
F	2
Total	**33**

[4] (a) Trace the min-heap merge sequence. Fill in each step.

Step	**Left node (weight)**	**Right node (weight)**	**New node weight**
1			
2			
3			
4			
5			

Initial min-heap: F(2), E(3), D(4), C(5), B(7), A(12)

Step 1: F(2) + E(3) → FE(5). Heap: D(4), C(5), FE(5), B(7), A(12)
Step 2: D(4) + C(5) → DC(9). Heap: FE(5), B(7), DC(9), A(12)
Step 3: FE(5) + B(7) → FEB(12). Heap: DC(9), A(12), FEB(12)
Step 4: DC(9) + A(12) → DCA(21). Heap: FEB(12), DCA(21)
Step 5: FEB(12) + DCA(21) → root(33).

[4] (b) Draw the resulting Huffman tree in the space below. Assign **0** to left edges and **1** to right edges.

Tree structure (0 = left, 1 = right):

Root(33): left FEB(12), right DCA(21)
FEB: left FE(5), right B(7)
FE: left F(2), right E(3)
DCA: left DC(9), right A(12)
DC: left D(4), right C(5)

Codes: F = 000 E = 001 B = 01 D = 100 C = 101 A = 11

[4] (c) Complete the encoding table and compute total bits.

Char	**Freq**	**Code**	**Code Length**	**Bits (Freq × Length)**
A	12			
B	7			
C	5			
D	4			
E	3			
F	2			
			Total bits (Huffman):	
			Total bits (fixed 3-bit encoding):	99
			Bits saved:	

A: 11, len=2, $12 \times 2 = 24$
B: 01, len=2, $7 \times 2 = 14$
C: 101, len=3, $5 \times 3 = 15$
D: 100, len=3, $4 \times 3 = 12$
E: 001, len=3, $3 \times 3 = 9$
F: 000, len=3, $2 \times 3 = 6$
Total Huffman: $24 + 14 + 15 + 12 + 9 + 6 =$ **80** bits. Fixed: $33 \times 3 = 99$ bits. Saved: **19 bits** (19.2% reduction).

[6] 31. **Ternary Search Tree vs. Standard Trie.**

[3] (a) A standard trie over a 26-character alphabet stores 500 words of average length 8. Estimate the worst-case number of child pointers allocated. How does a TST reduce this overhead, and how many pointers does a TST node hold?

Standard trie: each of the $500 \times 8 = 4{,}000$ nodes allocates 26 pointers. Worst case: $26 \times 4{,}000 = 104{,}000$ pointers, most of which are null.
A TST allocates exactly **3 pointers** per node (left, middle, right) regardless of alphabet size, giving at most $3 \times 4{,}000 = 12{,}000$ pointers — an $8.7\times$ reduction for this example.

[3] (b) A spell-checker must suggest words within one edit of a misspelled input. In two to three sentences, explain why a TST supports this better than either a standard trie or a hash table.

A standard trie requires exhaustive subtree traversal to find near-matches because siblings have no ordering that prunes the search. A hash table requires externally generating all edit-distance-1 candidates (up to $54L + 25$ strings for a word of length L), then hashing each one — correct, but wasteful. A TST's BST-ordered left/right children let the traversal naturally branch toward characters close to the current one, enabling bounded-edit-distance search without enumerating every candidate string first.

Section IV Challenge Questions

These questions explore deeper understanding. The instructor will determine which, if any, appear on the final version of this exam.

[10] 32. **Adversarial Hash Attacks.**

A web server stores HTTP request headers in a hash table using a deterministic, publicly known hash function. An attacker sends a crafted request with 500 headers, all chosen to hash to the same slot.

[4] (a) What is the expected lookup cost per header under normal conditions? What does it degrade to under the attacker's input, and why does the standard "expected $O(1)$" guarantee fail here?

Normal: $O(1)$ expected (good distribution, low α). Under the attack: all 500 headers land in one chain/cluster, making each lookup $O(n)$ and all 500 together $O(n^2)$. The "expected $O(1)$" guarantee is taken over the randomness of the hash function — if the function is deterministic and public, there is no randomness, so the expectation is meaningless against an adversary who can pre-compute the collision set.

[3] (b) Name the mitigation used by Python (since 3.3), Java, and Rust. How does it close the vulnerability?

Hash randomization: a random per-process seed is generated at startup and mixed into the hash function. Each run uses a different seed, so an attacker cannot pre-compute a collision set without knowing the seed, making adversarial attacks statistically infeasible.

[3] (c) Does hash randomization change the worst-case complexity? Justify your answer.

No. Worst-case complexity remains $O(n)$ — if the attacker learns the seed (e.g., via a timing side-channel), they can still craft collisions. Randomization makes attacks *impractical*, not impossible. The guarantee upgrades from "$O(1)$ expected under a random hash function" to "$O(1)$ expected with high probability per run" — a probabilistic, not absolute, worst-case bound.

[10] 33. **Amortized Analysis — Dynamic Array.**

A dynamic array doubles its capacity whenever it is full (starts at capacity 1).

[5] (a) Prove that the total cost of n append operations is $O(n)$ using the **aggregate method**. Sum the copy costs across all resize events.

Resize events occur when sizes reach 1, 2, 4, 8, …, n. The copy cost at each resize equals the current size. Total copy work across all resizes:

$$1 + 2 + 4 + \cdots + n \;=\; 2n - 1 \;<\; 2n \quad \text{(geometric series, ratio 2)}$$

Each of the n non-resize appends costs $O(1)$. Total: $n + 2n - 1 = O(n)$. Amortized cost per

append: $O(n)/n = O(1)$.

[3] (b) A student argues: "The dynamic array is not $O(1)$ per append because sometimes it takes $O(n)$." Explain the flaw in two sentences.

The student confuses *worst-case per individual operation* with *amortized cost over a sequence*. Amortized $O(1)$ means the total cost for *any* sequence of n operations is $O(n)$ — each expensive resize is pre-paid by the many cheap appends that preceded it (geometric doubling ensures each element is copied at most twice on average).

[2] (c) If the array grows by a fixed constant c slots instead of doubling, what is the total copy work? Is append still amortized $O(1)$?

Resize events occur every c appends. At the k-th resize, copy cost is kc. Total: $\sum_{k=1}^{n/c} kc = O(n^2/c) = O(n^2)$. Amortized cost per append: $O(n)$. No — fixed-increment growth **does not** achieve amortized $O(1)$; only constant-ratio doubling (or any factor > 1) does.

[8] 34. **Stability in Sort Pipelines.**

An HR system stores records with fields `(department, salary, hire_date)`. The following two-pass sort produces a department-then-salary report:

```
records.sort(key=lambda r: r.salary)        # pass 1: sort by salary
records.sort(key=lambda r: r.department)    # pass 2: sort by department
```

[4] (a) Explain why this produces the correct result *only if the second sort is stable*.

After pass 1, records are globally ordered by salary. Pass 2 groups records by department. If pass 2 is **stable**, records with the same department value preserve their relative order from pass 1 — i.e., they remain sorted by salary within each group. If pass 2 is **unstable**, equal-department records are shuffled arbitrarily, destroying the salary ordering within each department.

[2] (b) Python's `sort()` uses Timsort. Is this pipeline safe in Python? Name one language or sort algorithm where it would silently produce wrong output.

Safe in Python — Timsort is stable. It would silently fail if pass 2 used **heapsort** or an in-place **quicksort** (e.g., C's `qsort`, some C++ `std::sort` implementations), both of which are unstable. The output looks plausible but has incorrect intra-department salary ordering.

[2] (c) If the sort were unstable, describe a single-pass alternative that achieves the same correct result regardless of stability.

Sort once with a composite key:
`records.sort(key=lambda r:  (r.department, r.salary))`
This sorts by department first and uses salary as a tiebreaker in one pass — no stability assumption required.

[8] 35. **Load Factor and Clustering.**

[4] (a) The expected number of probes for an **unsuccessful** search under linear probing is:

$$\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$$

Compute this value at $\alpha = 0.50$, $\alpha = 0.75$, and $\alpha = 0.90$. Show your arithmetic.

$\alpha = 0.50$: $\frac{1}{2}(1 + \frac{1}{0.25}) = \frac{1}{2}(1+4) = \mathbf{2.5}$ probes.
$\alpha = 0.75$: $\frac{1}{2}(1 + \frac{1}{0.0625}) = \frac{1}{2}(1+16) = \mathbf{8.5}$ probes.
$\alpha = 0.90$: $\frac{1}{2}(1 + \frac{1}{0.01}) = \frac{1}{2}(1+100) = \mathbf{50.5}$ probes.

[4] (b) In three to four sentences, describe the self-reinforcing mechanism (primary clustering cascade) that explains why linear probing collapses near $\alpha = 1$ but separate chaining degrades much more gracefully.

In linear probing, a collision creates a contiguous run of occupied slots. Any key that hashes *anywhere into or immediately after* that run must walk the entire run before finding an empty slot, extending it. The extended run intercepts even more future keys, causing the cluster to grow faster than the fill rate alone would predict — a positive feedback loop. Separate chaining has no such feedback: each slot's linked list is independent, so a long chain at slot k has zero effect on lookups at slot $k+1$, and performance degrades linearly with α rather than hyperbolically.

[10] 36. **Container and Algorithm Selection.**

For each scenario, name the optimal container, the primary operation or algorithm that drives the choice, and provide a one-sentence justification.

Scenario	**Container**	**Operation / Algorithm**	**One-sentence justification**
Repeatedly remove the highest-priority job from a queue of 100k pending tasks.			
Check whether an ISBN appears in a catalog of 10 million books (no ordering needed).			
Return all indexed URLs that begin with a given domain prefix.			
Flag misspelled words and suggest the nearest correctly spelled word.			
Build an MST from a stream of weighted edges, detecting cycles cheaply as each edge arrives.			

Row 1: Binary Min-Heap / extract-min — $O(\log n)$ removal; array-backed heap is cache-friendly for heavy removal workloads.
Row 2: Hash Table / lookup — $O(1)$ expected; no prefix or ordering requirement, so trie overhead is unjustified.
Row 3: Trie (or Radix Tree) / prefix search — $O(P + K)$ to retrieve all K matches for a prefix of length P.
Row 4: Ternary Search Tree / bounded-edit-distance traversal — BST ordering on siblings enables near-match search without enumerating all candidates.
Row 5: Union-Find (DSU) / union + find — near-$O(1)$ amortized per operation; `find(u)==find(v)` detects cycles; this is exactly Kruskal's cycle-detection mechanism.

Multiple Choice Question Bank

Instructor use only — select questions to replace or supplement Section I.
Each question is worth 2 points. Questions are numbered independently from the exam.

[2] 1. A hash function computes $h(s)$ by summing the ASCII values of every character in string s. Which hash function property does this violate?

A. Determinism — the same input must always produce the same output

B. Speed — the function must run in $O(1)$

C. Avalanche effect — flipping one input bit must change ≈50% of output bits

D. Uniformity — each output slot must be equally likely for random input

Because the function sums character values, rearranging characters produces the same hash (`h("abc") == h("bca") == h("cab")`). A single-character change shifts the sum by at most one unit, changing almost none of the output bits. A function with the avalanche effect would scramble ≈50% of output bits for any one-bit change in input.

[2] 2. Most open-addressing hash table implementations trigger a resize when $\alpha = 0.75$ rather than waiting until $\alpha \to 1$. Which row of the probe-count table best explains why $\alpha = 0.75$ is already too high to ignore?

A. At $\alpha = 0.75$, the table is full and no more insertions are possible

B. At $\alpha = 0.75$, unsuccessful search already averages ≈8.5 probes under linear probing — 3.4× worse than at $\alpha = 0.5$

C. At $\alpha = 0.75$, separate chaining degrades to $O(n)$

D. At $\alpha = 0.75$, the hash function loses its avalanche property

From the linear-probing formula $\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$: at $\alpha = 0.50$: 2.5 probes; at $\alpha = 0.75$: 8.5 probes; at $\alpha = 0.90$: 50.5 probes. The threshold is set at 0.75 to stay left of the steep part of the curve, not at its bottom.

[2] 3. In a Huffman tree built from a frequency table, the character with the **highest** frequency receives:

A. The shortest code, because it sits closest to the root

B. The longest code, to distinguish it from all other characters

C. A code of the same length as every other character

D. A code determined by alphabetical order, not frequency

Huffman's greedy strategy always merges the two *lowest*-frequency nodes first, so high-frequency symbols remain near the root and accumulate fewer edge labels on their path — producing shorter codes. This is precisely why Huffman encoding minimizes total bits.

[2] 4. Which statement about **worst-case** hash table lookup is correct?

A. Open addressing is $O(1)$ worst-case; chaining degrades to $O(n)$

B. Chaining is $O(1)$ worst-case; open addressing degrades to $O(n)$

C. Both chaining and open addressing have $O(n)$ worst-case lookup

D. Both structures guarantee $O(\log n)$ worst-case via tree-structured buckets

In chaining, all n keys can hash to the same slot, forming one chain of length n. In open addressing, all n keys can cluster into the same contiguous run, requiring n probes to find an empty slot. In both cases, worst-case lookup is $O(n)$. The $O(1)$ guarantee is *expected*, not worst-case.

[2] 5. A hash table uses $h(k) = k \bmod m$ with $m = 2^{10} = 1024$. What specific problem does this non-prime table size introduce?

A. The hash function becomes non-deterministic for even-valued keys

B. The load factor can never exceed 0.5, wasting memory

C. Only the low 10 bits of k influence the hash; keys differing only in higher bits always collide

D. The table cannot support deletion under open addressing

$k \bmod 2^{10}$ extracts only the bottom 10 bits of k and ignores all higher bits. Keys like 1, 1025, 2049, ... all map to slot 1 regardless of their upper bits, causing heavy collisions for any key distribution with regularities in the low bits. A prime modulus mixes all bits of k into the slot index.

[2] 6. Under which condition is the Minimum Spanning Tree of a connected, undirected graph guaranteed to be **unique**?

A. When the graph is a complete graph (K_n)

B. When all edge weights are distinct

C. When the graph has no cycles (i.e., the graph itself is a tree)

D. When Prim's and Kruskal's produce the same set of edges

When all edge weights are distinct, the Cut Property uniquely identifies a single minimum crossing edge for every cut — so both Prim's and Kruskal's are forced to select the same edges. If any two edges share a weight, there may be multiple valid MSTs of equal total cost. Note: if the graph itself is already a tree, that tree *is* the unique MST trivially, but this is a special case of distinct effective weights, not a separate condition.

[2] 7. A network engineer is computing an MST for a complete graph with $V = 500$ vertices ($E \approx 125{,}000$ edges). Which algorithm is the better practical choice?

A. Prim's — on dense graphs, fewer priority-queue updates are needed relative to the edge count, and the $O((V+E)\log V)$ bound simplifies to $O(E \log V)$, which beats sorting all E edges

B. Kruskal's — Union-Find is always faster than a priority queue

C. Kruskal's — sorting edges is cheaper than maintaining a heap

D. Either — Prim's and Kruskal's have identical performance on all graphs

For dense graphs where $E \gg V$, Kruskal's $O(E \log E)$ sort dominates and is expensive. Prim's priority-queue approach performs $O(E)$ decrease-key operations at $O(\log V)$ each, giving $O(E \log V)$ total — often faster in practice on dense graphs. For sparse graphs ($E \approx V$), Kruskal's sort is cheap and Union-Find overhead is low, so Kruskal's is often preferred.

[2] 8. Match the algorithm to its time complexity (binary heap / comparison sort assumed).

A. Prim's: $O(E \log E)$ Kruskal's: $O((V+E) \log V)$

B. Prim's: $O((V+E) \log V)$ **Kruskal's:** $O(E \log E)$

C. Prim's: $O(V^2)$ Kruskal's: $O(V \log V)$

D. Prim's: $O(E)$ Kruskal's: $O(E + V)$

Prim's performs one extract-min and up to $\deg(v)$ decrease-key operations per vertex, giving $O((V+E) \log V)$ with a binary heap. Kruskal's sorts all edges once ($O(E \log E)$) then runs near-linear Union-Find operations. Since $E < V^2$, $\log E < 2 \log V$, so both are $\Theta(E \log V)$ on connected graphs — but the bound names differ by convention.

[2] 9. An in-memory symbol table is queried millions of times per second but updated (insert/delete) only once per minute. Which self-balancing BST is the better choice?

A. AVL tree — its stricter balance criterion ($|bf| \leq 1$) produces a shorter tree and faster search, which matters when reads dominate

B. Red-Black tree — fewer rotations per insert means lower write latency, which is always the bottleneck

C. Red-Black tree — it guarantees $O(1)$ search in all cases

D. AVL tree — it eliminates recoloring, reducing write overhead below Red-Black

AVL trees maintain height $\leq 1.44 \log n$; Red-Black trees allow height up to $2 \log n$. The shorter AVL tree means fewer comparisons per lookup — a meaningful advantage when reads vastly outnumber writes. When writes are frequent, the AVL overhead (more rotations per insert/delete) becomes the bottleneck, and Red-Black's looser balance with cheaper recoloring wins.

[2] 10. A student replaces a binary min-heap priority queue with a sorted linked list for a job scheduler that performs 50,000 insertions and 50,000 remove-min operations. What is the net impact?

A. No impact — both structures support priority-queue operations at the same cost

B. Both insert and remove-min degrade from $O(\log n)$ to $O(n)$

C. Insert degrades from $O(\log n)$ to $O(n)$ to maintain sorted order; remove-min improves from $O(\log n)$ to $O(1)$, but total work grows from $O(n \log n)$ to $O(n^2)$

D. Insert improves to $O(1)$; remove-min stays $O(\log n)$

A sorted linked list allows $O(1)$ remove-min (take the head) but requires $O(n)$ insert to maintain sorted order. With 50,000 insertions each costing up to $O(n)$: total $O(n^2)$. The heap gives $O(\log n)$ for both: total $O(n \log n)$. The container swap improved one operation but catastrophically worsened the other.

[2] 11. A Huffman encoder was built offline using character frequencies sampled from a large training corpus. Why does the encoder **not** need to recount character frequencies for each new file it compresses?

A. Huffman codes adapt automatically to each file's content at decode time

B. The codes are fixed by the tree structure, which was built from the training frequencies — the same tree (and therefore the same codes) is applied to every subsequent file without modification

C. Character frequencies are always uniform across files in the same domain

D. Recomputing frequencies would violate the prefix-free property

A Huffman tree is built once from a known frequency table. The resulting codes (paths from root to leaf) are then fixed. Any file can be encoded using those codes without revisiting frequencies — though the result is only *optimal* for files whose actual frequencies match the training distribution. This is the "static Huffman" model; "adaptive Huffman" (FGK algorithm) updates the tree on the fly for streaming input.

[2] 12. A trie stores n strings over an alphabet of size $|\Sigma|$. What is the time complexity of **searching** for a string of length k, regardless of n?

A. $O(n)$ — the trie must compare against every stored string

B. $O(\log n)$ — the trie uses binary-search-like branching

C. $O(k)$ — the search follows exactly k edges, one per character

D. $O(k \cdot |\Sigma|)$ — each node must scan all $|\Sigma|$ children

A trie search follows one edge per input character. At each node, the next character selects the child directly (array index or map lookup). The total number of steps is exactly the length of the query string, k, independent of how many strings n are stored — unlike a BST where cost grows with n.

[2] 13. Huffman coding, Prim's algorithm, and Kruskal's algorithm share which algorithmic classification?

A. Divide and conquer — they split the problem into independent halves recursively

B. Dynamic programming — they solve overlapping subproblems and cache results

C. Greedy — they make the locally optimal choice at each step, and it can be proven this produces a globally optimal result

D. Backtracking — they explore all candidate solutions before selecting the best

All three are greedy algorithms proven correct via an exchange argument: assuming a better solution exists and swapping edges (or symbols) leads to a contradiction. Huffman: always merge the two minimum-frequency nodes. Prim's: always extend via the minimum frontier edge. Kruskal's: always add the globally cheapest safe edge. None of them reconsider past choices or explore alternatives.

[2] 14. Unlike Huffman coding, Prim's, and Kruskal's, trie **insertion** and **search** are best characterized as:

A. Greedy — each character is chosen to minimize the remaining search space

B. Dynamic programming — results of sub-searches are cached to speed up queries

C. Deterministic traversal — the key's characters dictate a unique path through the tree with no choices or optimization involved

D. Divide and conquer — the trie splits the key space in half at each node

Trie operations follow a fixed, character-by-character path determined entirely by the input key. There is no optimization, no cached subproblem, and no branching strategy — each character maps to exactly one child at each level. A trie is a *data structure*; its operations are simple traversals, not instances of a higher-level algorithmic strategy.

[2] 15. Which test strategy most rigorously validates a hash table implementation against **worst-case** behavior?

A. Insert keys in sorted ascending order and verify that lookup time stays $O(1)$

B. Measure average lookup time over uniformly random keys at $\alpha = 0.50$

C. Construct a set of keys that all hash to the same slot and measure whether lookup time grows linearly with the number of colliding keys

D. Verify that the table resizes correctly when α crosses the threshold

Average-case testing with random keys only verifies expected behavior. To expose worst-case $O(n)$ degradation, you must construct adversarial inputs — keys engineered to collide at the same slot. For a public hash function, these are easy to compute; for a randomized one, you can still test by choosing keys that collide under the current seed. Resize testing and sorted-insertion testing exercise other code paths but do not reveal clustering or chain-length pathologies.